ERIC I RAMIREZ

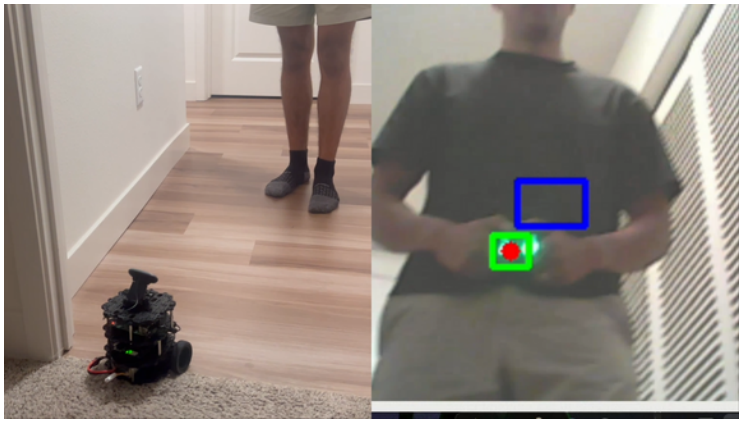
COMPUTER ENGINEER

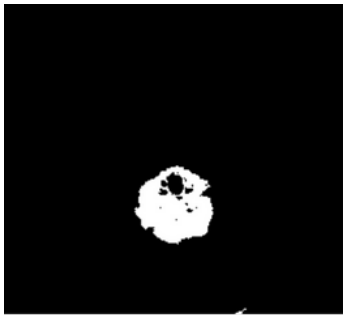
ericiramirez25@gmail.com

linkedin.com/in/eric-i-ramirez

(530) 744-4588

AUTONOMOUS IV POLE - CSU, CHICO





What?

- Programmed ROS2 (Python, OpenCV) on Raspberry Pi 4 + OpenCR for real-time motor control.
- Built LED-tracking pipeline with HSV masking, morphology, and safe-zone logic.
- Published overlay/mask topics in ROS2 for rapid tuning and debugging.

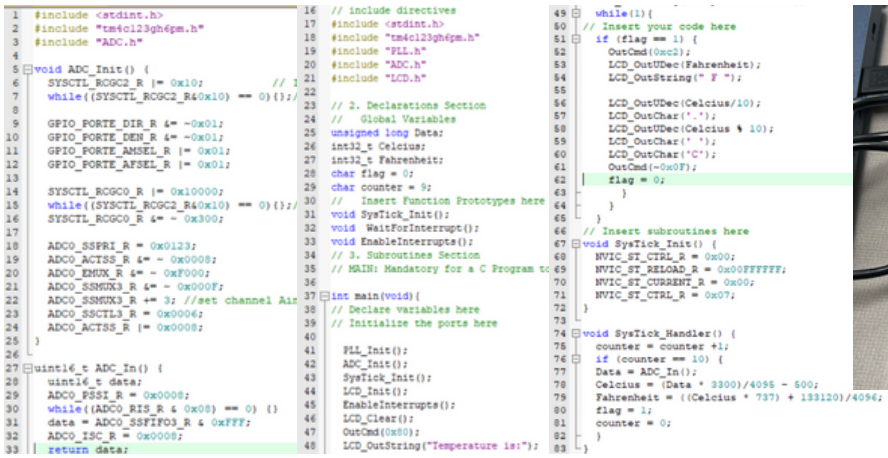
How?

- Designed vision-based robot to follow patients carrying an LED marker.
- Integrated TurtleBot3 Burger, Pi Camera, and LED hardware.

Results?

- Achieved stable tracking at 30 FPS and smooth motion up to 0.22 m/s.
- Cut debug/tuning time by 40% using live overlays.
- Presented at CSC^2 Symposium, demonstrating a medical robotics prototype with clinical potential.

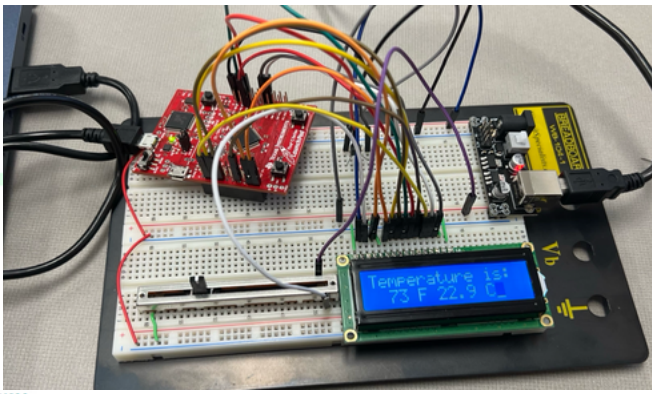
REAL-TIME TEMPERATURE SYSTEM - CSU, CHICO



```
#include <stdint.h>
#include "tm4c123gh6pm.h"
#include "ADC.h"

void ADC_Init() {
  SYSCTL_RCGC2_R |= 0x10;
  while((SYSCTL_RCGC2_R&0x10) == 0){};
  GPIO_PORTE_DIR_R &= ~0x01;
  GPIO_PORTE_DEN_R &= ~0x01;
  GPIO_PORTE_AMSEL_R |= 0x01;
  GPIO_PORTE_AFSEL_R |= 0x01;
  SYSCTL_RCGC0_R |= 0x10000;
  while((SYSCTL_RCGC2_R&0x10) == 0){};
  SYSCTL_RCGC0_R &= ~0x300;
  ADC0_SSPRI_R = 0x0123;
  ADC0_ACTSS_R &= ~0x0008;
  ADC0_EMUX_R &= ~0xF000;
  ADC0_SSMUX3_R &= ~0x000F;
  ADC0_SSMUX3_R += 3; //set channel Ain
  ADC0_SSCTL3_R = 0x0006;
  ADC0_ACTSS_R |= 0x0008;
}
uint16_t ADC_In() {
  uint16_t data;
  ADC0_PSSI_R = 0x0008;
  while((ADC0_RIS_R & 0x08) == 0) {}
  data = ADC0_SSFIFO3_R & 0xFFF;
  ADC0_ISC_R = 0x0008;
  return data;
```

```
// include directives
#include <stdint.h>
#include "tm4c123gh6pm.h"
#include "PLL.h"
#include "ADC.h"
#include "LCD.h"

// 2. Declarations Section
//   Global Variables
unsigned long Data;
int32_t Celcius;
int32_t Fahrenheit;
char flag = 0;
char counter = 9;
//   Insert Function Prototypes here
void SysTick_Init();
void  WaitForInterrupt();
void EnableInterrupts();
// 3. Subroutines Section
// MAIN: Mandatory for a C Program to

int main(void){
// Declare variables here
// Initialize the ports here

  PLL_Init();
  ADC_Init();
  SysTick_Init();
  LCD_Init();
  EnableInterrupts();
  LCD_Clear();
  OutCmd(0x80);
  LCD_OutString("Temperature is:");
```

```
  while(1){
// Insert your code here
  if (flag == 1) {
    OutCmd(0xc2);
    LCD_OutUDec(Fahrenheit);
    LCD_OutString(" F ");

    LCD_OutUDec(Celcius/10);
    LCD_OutChar('.');
    LCD_OutUDec(Celcius % 10);
    LCD_OutChar(' ');
    LCD_OutChar('C');
    OutCmd(~0x0F);
    flag = 0;
    }
  }
}
// Insert subroutines here
void SysTick_Init() {
  NVIC_ST_CTRL_R = 0x00;
  NVIC_ST_RELOAD_R = 0x00FFFFFF;
  NVIC_ST_CURRENT_R = 0x00;
  NVIC_ST_CTRL_R = 0x07;
}

void SysTick_Handler() {
  counter = counter +1;
  if (counter == 10) {
  Data = ADC_In();
  Celcius = (Data * 3300)/4095 - 500;
  Fahrenheit = ((Celcius * 737) + 133120)/4096;
  flag = 1;
  counter = 0;
  }
}
```



What?

- Programmed TM4C123 MCU in Embedded C with SysTick interrupts for real-time sampling.
- Configured 12-bit ADC and LCD driver using fixed-point arithmetic.
- Added a potentiometer dimmer to control LCD brightness.

How?

- Built a breadboard prototype with MCU, temp sensor, and LCD output.
- Implemented ADC + ISR code for periodic sampling in °C/°F.
- Integrated potentiometer control for adjustable LCD visibility.

Results?

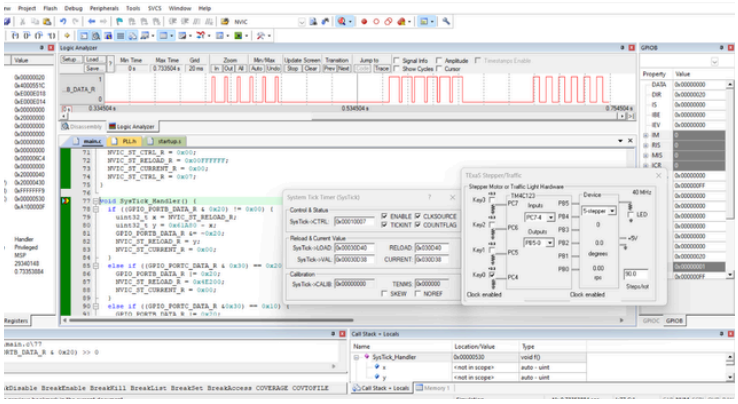
- Achieved ±0.1 °C resolution without floating-point overhead.
- Displayed real-time temps at 16 MHz update rate on the LCD.

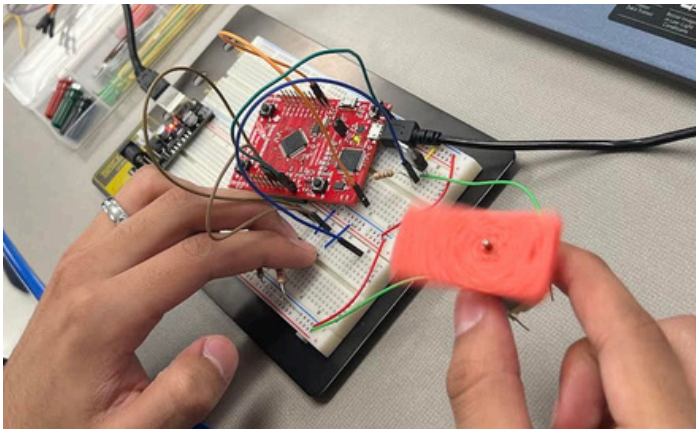
ERIC I RAMIREZ

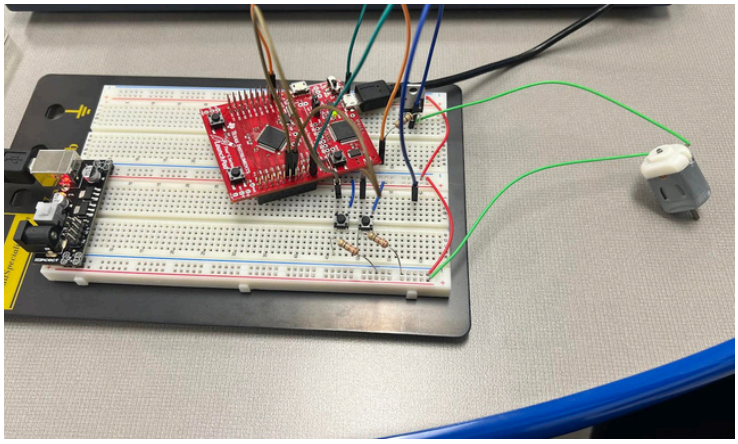
COMPUTER ENGINEER

ericiramirez25@gmail.com
linkedin.com/in/eric-i-ramirez
(530) 744-4588

DC MOTOR CONTROL SYSTEM - CSU, CHICO







What?

- Programmed SysTick PWM with interrupts to control a DC motor at 20%, 50%, and 80% speeds
- Built circuit to implement code and verify PWM motor control in hardware.

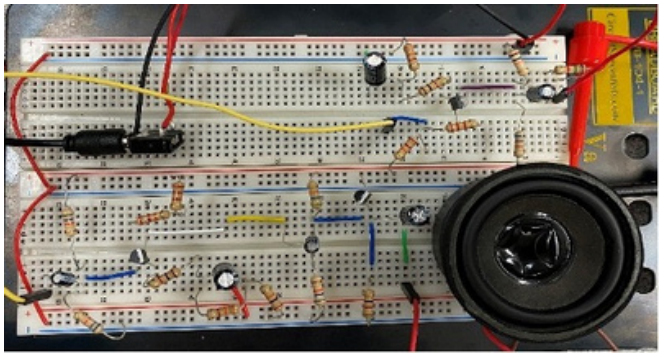
How?

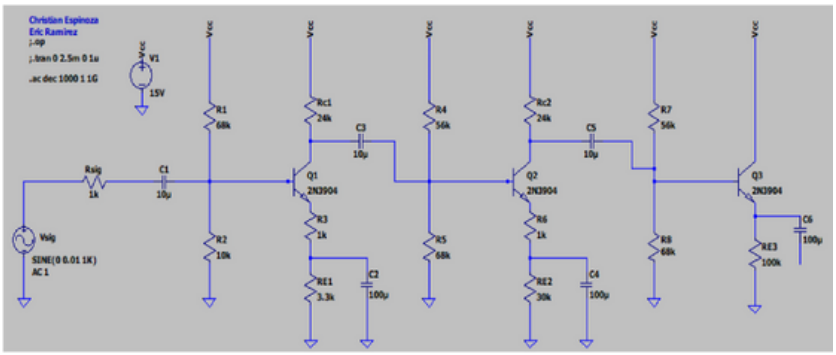
- Optimized MCU to 40 MHz and rewrote ISR for real-time RELOAD updates; fully tested using logic analyzer
- Configured timer registers and reload values to generate precise PWM signals

Results?

- Enabled 0% manual delay handling, improving speed control precision and reducing test time by 30%

THREE-STAGE BJT AUDIO AMPLIFIER - CSU, CHICO





What?

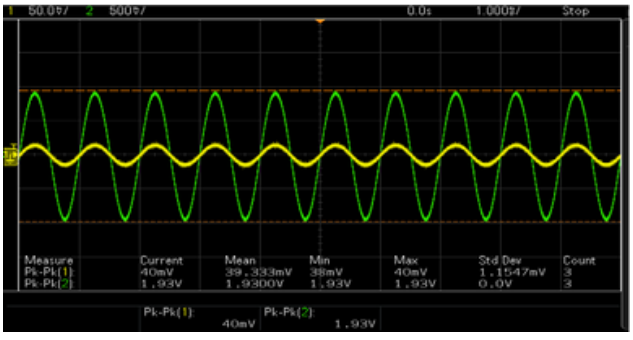
- Designed in LTSpice with biasing and bypass capacitors for stable cascaded gain.
- Built breadboard circuit, tuning values to fix clipping and distortion.

How?

- Implemented two common-emitter gain stages and one buffer stage.
- Images show LTSpice design, breadboard build, and oscilloscope output.

Results?

- Reached overall gain of 118x (~41 dB) into an 8 ohm speaker.
- Delivered clear audio across the 20 Hz–20 kHz range.

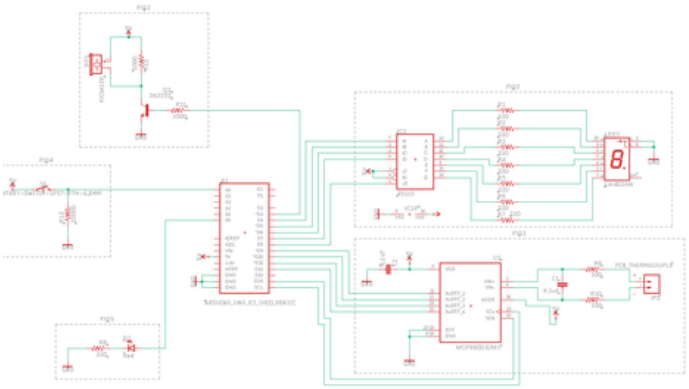


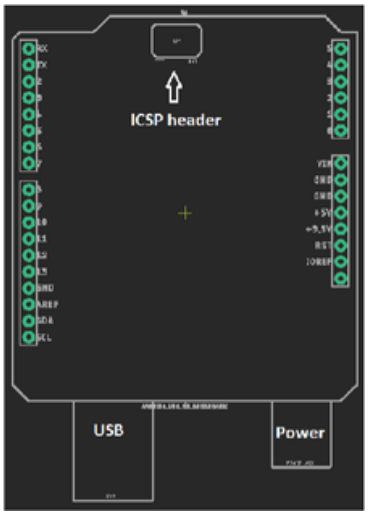
ERIC I RAMIREZ

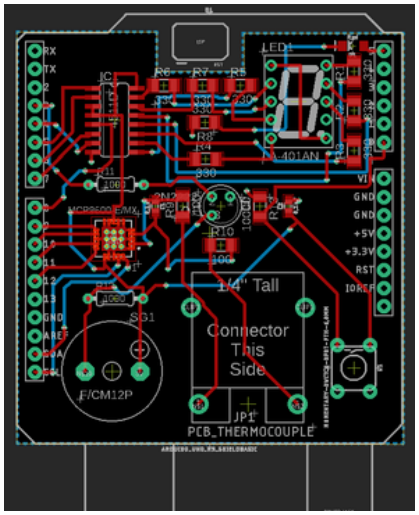
COMPUTER ENGINEER

ericiramirez25@gmail.com
linkedin.com/in/eric-i-ramirez
(530) 744-4588

PCB DESIGN - CSU, CHICO





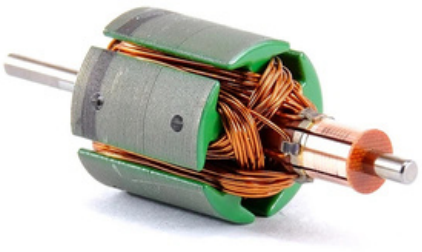


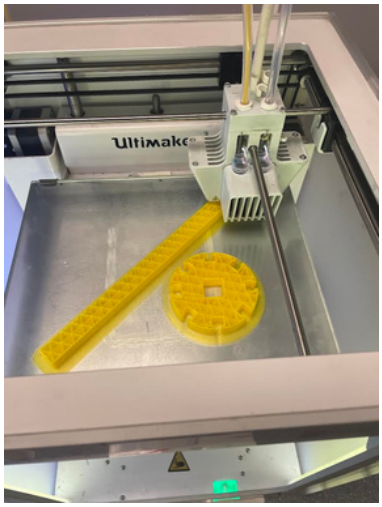
What?
- Designed a unified Arduino shield
- Created a PCB layout
- Integrated schematic sections into one design for the Arduino shield

How?
- Using AutoDesk(EAGLE) Software
- Integrated I2C, BCD, and alarm systems within strict shield layout specs

Results?
- Met 100% of design rules for trace width, spacing, and ground plane

DC MOTOR DESIGN - CSU, CHICO





What?
- Upscaled a factory DC Motor by 350%
- 3D printed model
- Improved efficiency

How?
- Using Solid Works to 3D print the shaft and housing & Soldered components together
- Ensured electric field aligned with magnetic poles

Results?
- Achieved continuous rotation at 3000 RPM
- Lifted 5.5lbs using a pully system